# Lecture 9 : Query Optimization

## RELATIONAL CALCULUS

ภาษา Non-procedural (บอกอยากได้อะไร ไม่ระบุ step การหา)

TRC (Tuple Relational Calculus): อ้างอิงทีละแถว (Tuple), { t | P(t) } (t คือตัวแปรแถว, P(t) คือเงื่อนไข) Ex.(หาพนักงานที่เงินเดือนสูงสุด): { t | t ∈ Employee ∧ ∀s(s ∈ Employee ⇒ t.Salary ≥ s.Salary) }

DRC (Domain Relational Calculus): อ้างอิงทีละคอลัมน์ (Column), { (d1,d2,...,dn) | P(d1,d2,...,dn) } Ex. หาชื่อพนักงานที่เงินเดือนสูงสุด: { (Name) | ∃ Salary,ID,DeptID (Employee(Name,ID,Salary,DeptID) ∧ ∀S,EID,EDeptID (Employee(EID,S,EDeptID) ⇒ Salary ≥ S)) }

## RELATIONAL ALGEBRA

ภาษา Procedural (ระบุ step งาน) ไว้สร้าง Execution Plan

**σ(Selection)**: กรองข้อมูลแถว (เหมือน WHERE), Ex.: σ Salary > 10000 ^ Dept='HR' (Employee)

**π(Projection)**: เลือกเฉพาะคอลัมน์ (เหมือน SELECT), Ex.: π name, dept (σ Salary > 1000000 (Employee))

**⋃(Union)/ − (Set Diff.)**: รวมเซต/ลบเซต (ตารางต้องโครงสร้างเหมือนกัน), Ex.: π Name (σ Dept='HR' (Employee)) ⋃ π Name (σ Dept='Finance' (Employee))

**× (Cartesian Product)**: จับคู่ทุกบรรทัดข้ามตาราง (ถ้า A มี 3 แถว, B มี 3 แถว จะได้ 9 แถว)

Retrieve Employees Who Earn the Highest Salary

TRC:
{t | t∈Employee ∧ ∀s(s∈Employee⇒t.Salary≥s.Salary)}

DRC:
{(Name) | ∃ Salary,ID,DeptID(Employee(Name,ID,Salary,DeptID) ∧ ∀S,EID,EDeptID(Employee(EID,S,EDeptID)⇒Salary≥S))}

## JOIN (⋈)

Natural Join (⋈): Join คอลัมน์ที่ชื่อเหมือนกันอัตโนมัติ

Theta Join (⋈θ): Join แบบระบุเงื่อนไขเอง (เช่น =, >, <) Ex.: Employee $\bowtie_{Employee.DeptID=Department.DeptID \wedge Department.DeptID=1}$ Department

Equi-Join (⋈=): Theta Join ที่ใช้เฉพาะเครื่องหมาย =

Outer Join: Left (⟕), Right (⟖), Full (⟗)

| DeptID | DeptName |
|---|---|
| 1 | HR |
| 2 | Finance |

| EmpID | Name | DeptID |
|---|---|---|
| 101 | Alice | 1 |
| 102 | Bob | 2 |
| 103 | Charlie | 1 |
| 104 | David | 3 |

Employee $\bowtie_{Employee.DeptID=Department.DeptID}$ Department

| EmpID | Name | DeptID | DeptName |
|---|---|---|---|
| 101 | Alice | 1 | HR |
| 102 | Bob | 2 | Finance |
| 103 | Charlie | 1 | HR |
| NULL | NULL | NULL | Marketing |

Employee⟕$_{Employee.DeptID=Department.DeptID}$Department

| EmpID | Name | DeptID | DeptName |
|---|---|---|---|
| 101 | Alice | 1 | HR |
| 102 | Bob | 2 | Finance |
| 103 | Charlie | 1 | HR |
| 104 | David | 3 | NULL |
| NULL | NULL | 4 | Marketing |

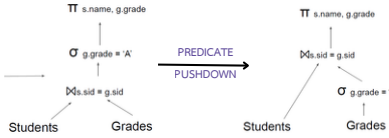
## QUERY OPTIMIZATION → Reduce Data Early

Predicate Pushdown: กรองแถว (σ) ก่อนนำไป Join (⋈) แบบช้า: π s.name, g.grade (σ g.grade = 'A' (students ⋈ grades)), แบบเร็ว (Pushdown): π s.name, g.grade (students ⋈ σ g.grade = 'A' (grades))

Projection Pushdown: เลือกเฉพาะคอลัมน์ (π) ที่ต้องใช้ตั้งแต่แรก

Use WHERE Instead of HAVING: WHERE กรองข้อมูลก่อนจับกลุ่ม (เร็วกว่า) ส่วน HAVING กรองหลังจับกลุ่ม

Avoid SELECT * : ระบุชื่อคอลัมน์ไปเลยเพื่อลดขนาด Data transfers, Ex.: เปลี่ยน SELECT * FROM STUDENTS... เป็น SELECT student_id FROM STUDENTS...




Optimizing Joins

- Join smaller tables first (เอาตารางเล็ก Join กันก่อน): Ex. ตาราง A=1000 แถว, B=400,000 แถว, C=500 แถว → วิธีที่ถูก: C ⋈ A ⋈ B หรือ A ⋈ C ⋈ B (เอา 500 ไป Join กับ 1000 ก่อน แล้วค่อยเอาผลลัพธ์ไป Join 400,000)

Partitioning Large Tables

- Horizontal Partitioning: หั่นคอลัมน์ (แก้ปัญหา Too many columns) Ex. ข้อมูลนักเรียน Students มีหลายคอลัมน์ → S1(ID, blah1-blah10), S2(ID, blah11-blah20)
- Vertical Partitioning: หั่นแถว (แก้ปัญหา Too many rows) Ex. ข้อมูลเยอะมาก A-Z เลยแยกแล้วแบ่งเป็น A-H, I-R, S-Z
- Replication: ก๊อปข้อมูลชุดเดียวกันไว้หลายเครื่อง (แก้ปัญหาเครื่องอื่นพัง แล้วเครื่องอื่นสามารถใช้งานต่อได้)

## INDEXING & DATABASE TUNE UP

Index

- ปัญหาถ้าไม่มี Index: DB ต้องค้นหาแบบสแกนทุกบรรทัด (Full Table Scan) ทำให้เกิด High Disk I/O และช้ามาก → วิธีแก้: สร้าง Index เก็บค่าคอลัมน์ + Pointer ชี้ไปที่แถว
- Ex. SQL: CREATE INDEX idx_employee_dept_salary ON Employee(DeptID, Salary);

เทคนิคจูนอัพระดับ Server (Single server environment):

- De-normalization: ยอมให้มีข้อมูลซ้ำซ้อนเพื่อลดจำนวนการ Join แต่ต้องระวังปัญหาตอน Insert/Update/Delete
- Precompute Aggregation: คำนวณค่าสถิติ (SUM, AVG) เตรียมไว้ล่วงหน้าเลย ไม่ต้องคำนวณใหม่ทุกครั้ง

คำสั่งเช็คการทำงานของ Query (Query Execution Plan):

- ใช้คำสั่ง EXPLAIN ANALYZE SELECT * FROM Employee WHERE Salary > 50000; เพื่อดูว่า DB มีแผนการดึงข้อมูลยังไง

# Lecture 10 : Multiuser Environment

## RELATIONAL CALCULUS

ACID Properties

- Atomicity: ต้องทำสำเร็จทั้งหมด หรือไม่ทำเลย
- Consistency: พา DB จาก Valid state หนึ่งไปอีก Valid state หนึ่ง
- Isolation: รันแยกจากกัน ไม่กวนกันจนกว่าจะจบ
- Durability: Commit แล้วข้อมูลอยู่ถาวรแม้ระบบล่ม

Read Anomalies

- Dirty Read: อ่านข้อมูลที่ transaction อื่นยังไม่ Commit (อาจถูก Rollback ภายหลัง)
- Non-Repeatable Read: อ่าน row เดิม 2 รอบได้ค่าไม่เท่ากัน (โดนคนอื่น Update ระหว่างนั้น)
- Phantom Read: Query เงื่อนไขเดิม 2 รอบได้จำนวน row ไม่เท่ากัน (โดนคนอื่น Insert เพิ่ม)

| Isolation Level | Allows Dirty Read | Allows Non-Repeatable Read | Allows Phantom Read |
|---|---|---|---|
| Read Uncommitted | ✅ Yes | ✅ Yes | ✅ Yes |
| Read Committed | ❌ No | ✅ Yes | ✅ Yes |
| Repeatable Read | ❌ No | ❌ No | ✅ Yes |
| Serializable | ❌ No | ❌ No | ❌ No |

Commands: START TRANSACTION; / COMMIT; (บันทึก) / ROLLBACK; (ย้อนกลับ) / SAVEPOINT sp1; (ตั้งจุดเซฟ) / ROLLBACK TO sp1; (ย้อนกลับแค่ถึงจุดเซฟ) *(ปล.: MySQL ปกติ Autocommit=1 เป็น default ทำให้ทุกคำสั่งคือ 1 transaction ต้องปิดเป็น 0 ก่อน)*

Locking

- Shared Lock (S): อ่านได้ แต่คนอื่นเขียนไม่ได้
- Exclusive Lock (X): เขียนได้อย่างเดียว คนอื่นอ่าน/เขียนไม่ได้
- SELECT ... FOR UPDATE: บังคับใช้ X Lock แบบ Row-level
- INSERT, UPDATE, DELETE: ใช้ X Lock อัตโนมัติ

ASSERTION & TRIGGER

- Assertion: Constraint ที่รับประกันว่าเงื่อนไขต้องเป็นจริงเสมอ แต่ออกแบบข้าม table ได้ (แต่หลาย DBMS รวมทั้ง MySQL ไม่รองรับ มักใช้ CHECK (salary <= 100000) แทน)
- Trigger: โค้ด auto-run เมื่อเกิด Event (INSERT, UPDATE, DELETE)
  - BEFORE: รันก่อน (ใช้ Validate/แก้ไขข้อมูลก่อนเซฟ)
  - AFTER: รันหลัง (ใช้ทำ Logging/Auditing)
  - ข้อจำกัด MySQL: รองรับแค่ Row-level (FOR EACH ROW), ห้ามสร้าง Trigger แบบเดียวกันซ้ำบนตารางเดียวกัน, และห้ามสั่ง COMMIT / ROLLBACK ข้างใน Trigger

PERMISSION

- สร้าง User: CREATE USER 'name'@'localhost' IDENTIFIED BY 'pass';
- ให้สิทธิ์: GRANT SELECT, INSERT ON mydb.emp TO 'name'@'localhost';
- ดึงสิทธิ์คืน: REVOKE INSERT ON mydb.emp FROM 'name'@'localhost';
- ใช้ FLUSH PRIVILEGES; เพื่ออัปเดตสิทธิ์

VIEWS

- สร้างจาก Query (CREATE VIEW v_name AS SELECT ...) ไม่ได้เก็บข้อมูลจริง (Virtual table)
- ข้อดี: ลดความซับซ้อน Query, ใช้ซ่อนข้อมูล (เช่น ไม่ select column เงินเดือนออกมาให้ Public view เห็น)
- Dynamic Role-Based View: สามารถใช้ WHERE CURRENT_USER() = ... เพื่อเช็คสิทธิ์คนล็อกอินให้เห็นข้อมูลต่างกันได้
- Updatable View: การสั่ง Update ผ่าน View จะทำได้ก็ต่อเมื่อ View นั้นสร้างจาก base table เดียว และไม่มี DISTINCT, GROUP BY, JOIN หรือ Aggregate functions (รวมถึงการใช้ SELECT ... FOR UPDATE ก็ทำได้เฉพาะกับ Updatable View เท่านั้น)

EXAMPLE CODE TRANSACTION & SAVEPOINT

```
SET autocommit = 0; -- ปิด autocommit (สำคัญ)
START TRANSACTION;
UPDATE accounts SET balance = balance - 500 WHERE account_id = 1; SAVEPOINT sp1; -- สร้างจุดเซฟ
UPDATE accounts SET balance = balance + 500 WHERE account_id = 2; ROLLBACK TO sp1; -- ย้อนกลับแค่ update ล่าสุด ไม่กระทบ account 1
COMMIT; -- ยืนยันการเปลี่ยน (or ROLLBACK; เพื่อยกเลิกทั้งหมด)
SET autocommit = 1;
```

EXAMPLE CODE SELECT ... FOR UPDATE (Row-Level Lock)

```
START TRANSACTION;
-- ล็อก row ที่ id=4 ไม่ให้คนอื่นแก้ จนกว่าเราจะ COMMIT
SELECT * FROM sales_team WHERE id = 4 FOR UPDATE;
UPDATE sales_team SET position = 'Senior' WHERE id = 4;
COMMIT;
```

EXAMPLE CODE TRIGGER (เช็คเงื่อนไขก่อน Insert)

```
-- สร้าง Trigger ทำงาน "ก่อน" Insert ข้อมูลลงตาราง employee
CREATE TRIGGER salary_check_before_insert
BEFORE INSERT ON employee FOR EACH ROW
BEGIN -- เช็คค่าใหม่ที่กำลังจะถูก Insert โดยใช้ NEW.ชื่อคอลัมน์
    IF NEW.salary > 100000 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Salary exceeds the limit!';
    END IF; END;
-- ถ้าเป็น AFTER INSERT มักใช้ทำ Log เช่น
-- INSERT INTO log_table VALUES(NEW.id, 'INSERTED');
```

EXAMPLE CODE DYNAMIC ROLE-BASED VIEW

```
-- View ที่ดึงข้อมูลตามสิทธิ์ของคนที่ Login อยู่ (CURRENT_USER)
CREATE VIEW dept_emp_view AS
SELECT e.id, e.name, e.department FROM employee e
JOIN manager m ON e.department = m.department
WHERE CURRENT_USER() = CONCAT(m.username, '@localhost');
```

EXAMPLE CODE PERMISSION

```
-- สร้าง User
CREATE USER 'student'@'localhost' IDENTIFIED BY 'pass123';
-- ให้สิทธิ์ SELECT, INSERT เฉพาะตาราง employee
GRANT SELECT, INSERT ON mydb.employee TO 'student'@'localhost';
-- ดึงสิทธิ์ INSERT คืน
REVOKE INSERT ON mydb.employee FROM 'student'@'localhost';
FLUSH PRIVILEGES; -- บังคับรีเฟรชสิทธิ์ (ต้องมี)
```


**วิศิษฐ์ สุวรรณเนาว์ 67070501042**


# Lecture 11 : NoSQL & MongoDB

## NOSQL CONCEPT

- Not Only SQL: ไม่ได้มาแทน SQL แต่ใช้กับงานที่ SQL ไม่ถนัด
- จุดเด่น: Schema-less (โครงสร้างยืดหยุ่น ไม่ต้องฟิกซ์คอลัมน์), Horizontally Scalable (ขยายเซิร์ฟเวอร์แนวนอนได้ง่าย), เน้น Read/Write เร็ว
- Data Models (ประเภทของ NoSQL):
  1. Document: เก็บเป็น JSON/BSON (เช่น MongoDB, Firebase) - เหมาะกับ User profiles, Catalog
  2. Key-Value: เก็บคำผูกกับคีย์ (เช่น Redis) - เหมาะกับ Caching, Session
  3. Column-Family: (เช่น Cassandra) - เหมาะกับเก็บ Log, Time-series
  4. Graph: โหนดและเส้นเชื่อม (เช่น Neo4j) - เหมาะกับ Social network, Recommendation

## MONGODB STRUCTURE

- Database : Database
- Collection : Table (แต่เป็น Schema-less)
- Document : Row (เก็บข้อมูลเป็น BSON/JSON แบบซ้อนกันได้)
- Field : Column (เก็บแบบ Key: Value)

## MONGODB CRUD OPERATIONS

```
// --- Database & Collection ---
show dbs;                          // ดู DB ทั้งหมด
use myDB;                          // สร้าง/สลับไปใช้ DB ชื่อ myDB
show collections;                  // ดูตารางทั้งหมด
db.createCollection("emps"); // สร้าง Collection ชื่อ emps
db.emps.drop();                    // ลบ Collection
// --- C: CREATE (เพิ่มข้อมูล) ---
db.emps.insertOne({ name: "Alice", age: 21 });
db.emps.insertMany([{ name: "Bob" }, { name: "Cat" }]);
// --- R: READ (ค้นหา) ---
db.emps.find();    // ดึงทั้งหมด (เหมือน SELECT *)
db.emps.find({ age: { $gt: 20 } });   // $gt = มากกว่า 20
// (Op: $eq(=), $gt(>), $gte(>=), $lt(<), $lte(<=), $ne(!=))
// --- U: UPDATE (แก้ไข) ---
// พารามิเตอร์: ({ เงื่อนไขค้นหา }, { $set: { ค่าใหม่ } })
db.emps.updateOne({ name: "Alice" }, { $set: { age: 22 } });
db.emps.updateMany({}, { $inc: { age: 1 } }); // $inc = เพิ่ม 1
// --- D: DELETE (ลบ) ---
db.emps.deleteOne({ name: "Alice" });
db.emps.deleteMany({ age: { $lt: 18 } });
```

## MERN STACK & EXPRESS.JS (BACKEND)

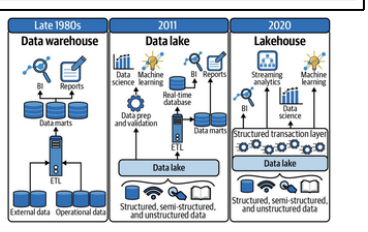
- MERN Stack: MongoDB + Express.js (Web framework) + React / HTML+JS + Node.js (รันใหม่)
- Mongoose (ODM): เป็นไลบรารีที่ช่วยเพิ่ม Schema ให้ MongoDB (บังคับโครงสร้าง/ตรวจสอบ Type) ทำให้ทำงานปลอดภัยขึ้น
- Express Routing: METHOD + PATH → HANDLER (เช่น GET /users → ส่งข้อมูล user, POST/users → รับข้อมูล user)
- Middleware: ฟังก์ชันที่ดักทำงานก่อนจะไปถึง Route จริง ๆ (เช่น express.json() ใช้แปลง body เป็น JSON)
- กฎของ JSON: Key ต้องใส่ Double Quotes ("key"), ไม่มีลูกน้ำ (,) ปิดท้ายตัวสุดท้าย, และ Case-sensitive
- Request Lifecycle: User กด Form → Frontend ส่ง HTTP → Express Backend รับเรื่อง → คุยกับ MongoDB (ด้วย Mongoose) → ส่ง Response กลับให้ User

| ประเภท NoSQL | Structure | ตัวอย่าง DB | Best For |
|---|---|---|---|
| Document | เป็นก้อน JSON / BSON | MongoDB, Firebase | User profiles, Catalog สินค้า, Content |
| Key-Value | จับคู่ Key -> Value | Redis, DynamoDB | Caching, เก็บ Session, ตัวนับแบบ Real-time |
| Column-Family | แถวจัดกลุ่มตาม Column | Cassandra, HBase | เก็บ Log events, Time-series, Data analytics |
| Graph | Node + Edge | Neo4j | Social networks, Recommendation |

EXAMPLE CODE MongoDB (Document Structure)

```
// Database: "University"
//    └──── Collection: "students" (เหมือนตาราง)
//          └──── Document (แต่ละก้อนคือ 1 Row)
{
  "_id": ObjectId("drvvf34f4"), // สร้างให้อัตโนมัติ (Primary Key)
  "name": "Alice",              // Field (String)
  "age": 21,                    // Field (Number)
  "isStudent": true,            // Field (Boolean)
  "skills": ["Python", "MongoDB"],  // Field (Array)
  "address": {                  // Field (Object ซ้อนใน Object)
    "city": "Bangkok",
    "zip": "10110"
  }
}
```

| หัวข้อ (Feature) | SQL (Relational) | NoSQL (Non-Relational) |
|---|---|---|
| Schema (โครงสร้าง) | โครงสร้างตายตัว (DB-enforced) เปลี่ยนยาก | ยืดหยุ่น (App-enforced) ใส่ฟิลด์เพิ่มได้เลย |
| รูปแบบข้อมูล (Format) | ตาราง (Table: Rows & Columns) | Document, Key-Value, Graph, Column |
| การขยายระบบ (Scaling) | Vertical (อัปสเปคเครื่องเดิม) | Horizontal (ซื้อเครื่องใหม่มาต่อเพิ่มง่าย) |
| ความโดดเด่น | ข้อมูลซับซ้อน (Complex Joins), ทำ Transactions แม่นยำ (ACID) | รับโหลดคนเยอะ, Big Data, โครงสร้างเปลี่ยนบ่อย (Agile/MVPs) |
| เมื่อไหร่ควรใช้? | ระบบบัญชี, ธุรกรรมการเงิน | แชท (Real-time), IoT, Social Network, แคชชิง |




# Lecture 12 : Advanced Topics

## OLTP & OLAP

| Feature | OLTP (Online Transaction Processing) | OLAP (Online Analytical Processing) |
|---|---|---|
| Purpose | จัดการธุรกรรมรายวันแบบ Real-time | รองรับการวิเคราะห์ข้อมูลซับซ้อนและการตัดสินใจ |
| Operations | Insert, Update, Delete, Simple Queries | Complex Queries, Aggregations (สรุปผล), Data Mining |
| Data Freshness | อัปเดตล่าสุดเสมอ (Always up-to-date / real-time) | ข้อมูลในอดีต (Historical) หรืออัปเดตเป็นรอบๆ |
| Transaction Type | สั้น, เกิดขึ้นบ่อย, ทำงานแบบ Atomic (เสร็จสมบูรณ์ในตัว) | ยาว, ซับซ้อน, เน้นอ่านข้อมูลปริมาณมาก (Read-heavy) |
| Schema Design | Highly normalized (ลดความซ้ำซ้อนสุด) | Denormalized (ยอมให้ซ้ำซ้อน เช่น Star / Snowflake Schema) |
| Data Volume | ค่อนข้างเล็ก (Relatively small) ต่อ 1 ธุรกรรม | ใหญ่มากระดับ Millions/Billions of records |
| Users | ลูกค้า, พนักงานขาย, พนักงานปฏิบัติการ | นักวิเคราะห์ (Analysts), ผู้บริหาร, ผู้จัดการ |
| Performance Focus | ความเร็ว Read/Write และ Concurrency (คนเข้าพร้อมกัน) | ความเร็วในการ Read และการทำ Aggregation ซับซ้อน |
| Backup/Recovery | สำคัญมากระดับวิกฤต (Critical) สำหรับทุกๆ ธุรกรรม | สำคัญน้อยกว่าในระดับ Query มักใช้การ Backup แบบ Batch (เป็นรอบๆ) |
| Example Systems | MySQL, PostgreSQL, Oracle | Microsoft SSAS, Oracle OLAP, Snowflake |

## DISTRIBUTED DATABASE

- Horizontal Partitioning (Sharding): แบ่งข้อมูลตาม Row ไปไว้คนละเซิร์ฟเวอร์ (เช่น แบ่งตามภูมิภาค)
- Vertical Partitioning: แบ่งข้อมูลตาม Column (เช่น แยกคอลัมน์ประวัติการซื้อไปไว้อีกเซิร์ฟเวอร์)
- Replication: ก๊อปปี้ข้อมูลให้เหมือนกันเป๊ะๆ หลายเซิร์ฟเวอร์ (ช่วยเรื่อง Fault tolerance ถ้าเซิร์ฟเวอร์พังยังมีสำรอง)
- CAP Theorem: ทฤษฎีข้อจำกัดของระบบ Distributed ที่ต้องเลือกได้อย่างมาก 2 อย่างจาก: Consistency (ข้อมูลตรงกัน), Availability (ระบบไม่ล่ม), Partition Tolerance (เครือข่ายขาดก็ยังทำงานได้)

## CLOUD COMPUTING & CHARAC.

Networking -> Storage -> Servers -> Virtual -> OS -> Middleware -> Runtime -> Data -> App

Responsibility

- On-Premise: เราดูแลเองทั้งหมด 100%
- IaaS (Infrastructure): เราดูแลตั้งแต่ OS → App (เขาดูแลแค่เซิร์ฟเวอร์/เน็ตเวิร์คให้)
- PaaS (Platform): เราดูแลแค่ Data กับ App (เขาเตรียม OS/Runtime ให้หมดแล้ว)
- SaaS (Software): เขาดูแลให้ 100% (เราแค่ใช้งาน เช่น Gmail, Google Drive)

Key Characteristics

- Availability: ระบบใช้งานได้ตลอดเวลา (เช่น Uptime 99.99%)
- Scalability: ขยายระบบรองรับการเติบโต
  - Vertical: อัปสเปคเครื่องเดิมให้แรงขึ้น
  - Horizontal: ซื้อเครื่องสเปคเท่าเดิมมาต่อเพิ่ม (Scale out)
- Elasticity: ความยืดหยุ่น หด/ขยาย สเปคได้อัตโนมัติตามทราฟฟิก (Auto-scale)
- Fault Tolerance: ป้องกันระบบล่มเมื่อมีชิ้นส่วนพัง
- Disaster Recovery: กู้ข้อมูลเมื่อมีหายนะข้อมูลหาย

## DB & DATAWAREHOUSE & DATALAKE

| Aspect | Database | Data Warehouse | Data Lake |
|---|---|---|---|
| Purpose | เก็บและจัดการข้อมูลปฏิบัติการปัจจุบัน (Transactions, records) | เก็บข้อมูลในอดีตที่ผ่านการคลีนแล้ว (Cleaned data) เพื่อวิเคราะห์ | เก็บข้อมูลดิบ หลากหลายรูปแบบ เพื่อความยืดหยุ่นในการใช้ในอนาคต |
| Data Type | Structured | Structured | Structured, Semi-structured, Unstructured |
| Data Storage | Normalized และรองรับ Transactional | ผ่านกระบวนการ Preprocessed, Aggregated, Structured | ดิบและยังไม่ผ่านการประมวลผล (Raw and unprocessed) |
| Schema | เข้มงวด บังคับตอนเขียนข้อมูลลงไป (Schema-on-write) | เข้มงวด บังคับตอนเขียนข้อมูลลงไป (Schema-on-write) | ยืดหยุ่น นำมาครอบเฉพาะตอนจะอ่านข้อมูล (Schema-on-read) |
| Query Type | สั้น, เร็ว (CRUD operations) | Complex analytical queries (Aggregations, Joins) | Custom queries หลังจากทำการสำรวจข้อมูลแล้ว (Data exploration) |
| Use Case | รันระบบปฏิบัติการธุรกิจ (OLTP) | ทำ Business Intelligence (BI), Dashboards, Reporting (OLAP) | Big data analytics, AI/ML, การเก็บข้อมูลระยะยาว (Archival) |
| Performance Focus | ความเร็วของ Transaction และ Concurrency | ความเร็วของ Query สำหรับการวิเคราะห์สเกลใหญ่ | ความสามารถในการขยาย Storage และความยืดหยุ่น |
| Cost | ปานกลาง (Moderate) | สูงกว่า (เพราะ Optimized มาเพื่อ Performance) | ต่ำกว่า (เพราะ Optimized มาเพื่อการเก็บข้อมูลยอะๆ Storage ถูกๆ) |
| Example Technologies | MySQL, PostgreSQL, Oracle DB | Snowflake, Redshift, BigQuery | Hadoop HDFS, AWS S3, Azure Data Lake |

# Practice Exercise M3 EXAM

EMPLOYEE

| EmployeeID | username | Department | HireYear |
|---|---|---|---|
| 201 | Somchai | Engineering | 2020 |
| 202 | Suda | IT | 2021 |
| 203 | Anan | Engineering | 2022 |
| 204 | Malee | Data | 2021 |
| 205 | Niran | IT | 2023 |

TRAINING

| EmployeeID | CourseID | CourseName | Department | Hours |
|---|---|---|---|---|
| 201 | T101 | Database | Engineering | 10 |
| 201 | T102 | System | Engineering | 8 |
| 202 | T103 | Web | IT | 6 |
| 202 | T105 | Network | IT | 7 |
| 203 | T101 | Database | Engineering | 10 |

- SQL Query ข้อมูล EmployeeID, EmployeeName, และ HireYear ของพนักงานทั้งหมด โดยเรียงตาม HireYear จากใหม่ไปเก่า

```
SELECT EmployeeID, EmployeeName, HireYear
FROM EMPLOYEE ORDER BY HireYear DESC;
```

- SQL Query ข้อมูล CourseID, CourseName, TotalEmployees ซึ่งเป็นพนักงานที่เข้าอบรมในแต่ละหลักสูตร

```
SELECT CourseID, CourseName, COUNT(EmployeeID)
AS TotalEmployees FROM TRAINING GROUP BY
CourseID, CourseName;
```

- SQL Query ข้อมูล EmployeeID, EmployeeName และ TotalHours ซึ่งเป็นผลรวมชั่วโมงการอบรมของพนักงานแต่ละคน

```
SELECT E.EmployeeID, E.EmployeeName,
SUM(T.Hours) AS TotalHours
FROM EMPLOYEE E
LEFT JOIN TRAINING T ON E.EmployeeID =
T.EmployeeID
GROUP BY E.EmployeeID, E.EmployeeName;
```

- SQL Query ข้อมูล EmployeeID, EmployeeName และ TotalHours ของพนักงานที่มีชั่วโมงการอบรมรวมมากกว่า 15 ชั่วโมง

```
SELECT E.EmployeeID, E.EmployeeName,
SUM(T.Hours) AS TotalHours
FROM EMPLOYEE E
JOIN TRAINING T ON E.EmployeeID = T.EmployeeID
GROUP BY E.EmployeeID, E.EmployeeName
HAVING SUM(T.Hours) > 15;
```

- SQL Query ข้อมูล EmployeeID, EmployeeName และ TotalHours โดยเลือกเฉพาะพนักงานที่มี TotalHours มากกว่าพนักงานทุกคนในแผนก IT และแผนก Data

```
SELECT E.EmployeeID, E.EmployeeName,
SUM(T.Hours) AS TotalHours
FROM EMPLOYEE E
JOIN TRAINING T ON E.EmployeeID = T.EmployeeID
GROUP BY E.EmployeeID, E.EmployeeName
HAVING SUM(T.Hours) > ALL (
    SELECT SUM(Hours) FROM TRAINING
    WHERE Department IN ('IT', 'Data')
    GROUP BY EmployeeID);
```

- SQL Query ข้อมูล EmployeeID, EmployeeName ของพนักงานที่เข้าอบรมในหลักสูตรอย่างน้อย 1 หลักสูตร ที่อยู่ในแผนกเดียวกับหลักสูตรที่มีจำนวนพนักงานเข้าอบรมมากที่สุด

```
SELECT DISTINCT E.EmployeeID, E.EmployeeName
FROM EMPLOYEE E
JOIN TRAINING T ON E.EmployeeID = T.EmployeeID
WHERE T.Department = (SELECT Department
    FROM TRAINING
    GROUP BY CourseID, Department
    ORDER BY COUNT(EmployeeID) DESC LIMIT 1);
```

- TRC, DRC, RA Query : Find EmployeeID of employees who attended at least one training course.

**RA:** $\pi_{\text{EmployeeID}}$ (TRAINING)

**TRC:** {t.EmployeeID | t∈TRAINING}

**DRC:** {eID | ∃ cID, cN, dept, hrs (TRAINING(eID, cID, cN, dept, hrs))}

- TRC, DRC, RA Query : Find EmployeeName of employees who attended the course "Database" and "Web Dev".

**RA:** $\pi_{\text{EmployeeName}}$ (EMPLOYEE ⋈ ($\pi_{\text{EmployeeID}}$ ($\sigma_{\text{CourseName='Database'}}$ (TRAINING)) ∩ $\pi_{\text{EmployeeID}}$ ($\sigma_{\text{CourseName='Web Dev'}}$ (TRAINING))))

**TRC:** {e.EmployeeName | e∈EMPLOYEE ∧ ∃ t1 ∈ TRAINING (e.EmployeeID = t1.EmployeeID ∧ t1.CourseName = 'Database') ∧ ∃t2∈TRAINING (e.EmployeeID = t2.EmployeeID ∧ t2.CourseName = 'Web Dev') }

**DRC:** {eN | ∃ eID, eD, hY (EMPLOYEE(eID, eN, eD, hY) ∧ ∃ cID1, tD1, h1 (TRAINING(eID, cID1, 'Database', tD1, h1)) ∧ ∃ cID2, tD2, h2 (TRAINING(eID, cID2, 'Web Dev', tD2, h2)))}

## PREPARE LECTURE 9-12 (ADDITIONAL) FOR M3 EXAM

QUERY EXECUTION PLAN

- Access Methods → Join Strategies → Operators → Cost Estimation
- SQL Query → Multiple Possible Plans → Lowest Estimated Cost

QUERY EXECUTION PROCESS

- Parsing → Binding (Semantic Checking) → Optimization: query optimizer → Execution → Result Output

COMPONENTS OF QUERY EXECUTION PLAN

- **Accessing Data:** Full Table Scan, Index Scan, Index Seek
- **Combining Tables:** Nested Loop Join, Hash Join, Merge Join
- **Ordering and Summarizing Data:** Sorting, Aggregation
- **Reducing Output Size** (Important!)**:** Selection (σ), Projection (π):

TYPES OF INDEXES

1. Primary Index (Clustered Index), 2. Secondary Index (Non-Clustered Index), 3. Unique Index, 4. Full-Text Index, 5. Partial Index, 6. Bitmap Index, 7. Covering Index, 8. Hash Index

## EXAMPLE NORMALIZATION

- **Table 2 Doctor-Patient-Appointment**

| Doctor Name | Specialty | Office Phone | Mobile Phone | Patient Name | Diagnosis | Treatment | Insurance | Hospital | Building | Floor | Room Number | Appointment Date |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Dr. John Smith | Cardiology | 123-456-7890 | 987-654-3210 | Alice Johnson | Hypertension | Prescribed medication... | Aetna | City General Hospital | North Wing | 3 | 305A | 2024-02-17 |
| Dr. Emily Davis | Dermatology | 234-567-8901 | 876-543-2109 | Robert Brown | Eczema | Topical steroid cream | Blue Cross | Metro Medical Center | East Wing | 2 | 204B | 2024-02-18 |
| Dr. John Smith | Cardiology | 123-456-7890 | 987-654-3210 | Michael Carter | High Cholesterol | Prescribed statins | United Healthcare | City General Hospital | North Wing | 3 | 305A | 2024-02-19 |
| Dr. Emily Davis | Dermatology | 234-567-8901 | 876-543-2109 | Jessica Miller | Acne | Oral antibiotics and skincare | Cigna | Metro Medical Center | East Wing | 2 | 204B | 2024-02-20 |

1NF



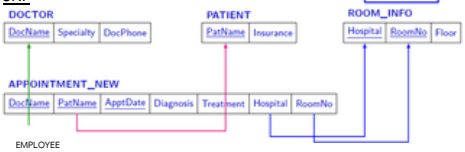

2NF







3NF




EMPLOYEE

| EmployeeID | EmployeeName | EmpDepartment | HireYear |
|---|---|---|---|
| 201 | Somchai | Engineering | 2020 |
| 202 | Suda | IT | 2021 |
| 203 | Anan | Engineering | 2022 |

TRAINING

| EmployeeID | CourseIDs | CourseNames | Hours |
|---|---|---|---|
| 201 | T101, T102 | Database, System Design | 10, 8 |
| 202 | T103, T105 | Web Dev, Network | 6, 7 |
| 203 | T101 | Database | 10 |

**1NF CHECK**

ตารางที่สอง TRAINING_RAW ไม่ผ่าน 1NF เพราะมีหลายค่าใน cell เดียว เช่น CourseIDs = T101, T102, CourseNames = Database, System Design, Hours = 10, 8

แก้เป็น 1NF: ต้องทำให้หนึ่ง cell มีหนึ่งค่า และหนึ่ง row แทนหนึ่งการอบรมของพนักงานหนึ่งคนในหนึ่ง course

**2NF & 3NF CHECK**

ตารางชุดนี้ ยังไม่ผ่าน 2NF เพราะตาราง TRAINING มี composite key = {EmployeeID, CourseID} แต่มี attribute ที่ขึ้นกับแค่ CourseID เท่านั้น

แก้เป็น 2NF:

- EMPLOYEE_2NF(EmployeeID, EmployeeName, EmpDeptID, EmpDeptName, HireYear)
- COURSE_2NF(CourseID, CourseName, CourseDeptID, CourseDeptName, Hours)
- TRAINING_2NF(EmployeeID (FK), CourseID (FK), Result)

EMPLOYEE

| EmployeeID | EmployeeName | EmpDeptID | EmpDeptName | HireYear |
|---|---|---|---|---|
| 201 | Somchai | D01 | Engineering | 2020 |
| 202 | Suda | D02 | IT | 2021 |
| 203 | Anan | D01 | Engineering | 2022 |
| 204 | Malee | D03 | Data | 2021 |

COURSE

| CourseID | CourseName | CourseDeptID | CourseDeptName | Hours |
|---|---|---|---|---|
| T101 | Database | D01 | Engineering | 10 |
| T102 | System Design | D01 | Engineering | 8 |
| T103 | Web Dev | D02 | IT | 6 |
| T105 | Network | D02 | IT | 7 |

TRAINING

| EmployeeID | CourseID | CourseName | CourseDeptID | Hours | Result |
|---|---|---|---|---|---|
| 201 | T101 | Database | D01 | 10 | Pass |
| 201 | T102 | System Design | D01 | 8 | Pass |
| 202 | T103 | Web Dev | D02 | 6 | Pass |
| 202 | T105 | Network | D02 | 7 | Fail |
| 203 | T101 | Database | D01 | 10 | Pass |

ตารางชุดนี้ ยังไม่ผ่าน 3NF เพราะมี transitive dependency:

- EmployeeID -> EmpDeptID และ EmpDeptID -> EmpDeptName
- CourseID -> CourseDeptID และ CourseDeptID -> CourseDeptName

แก้เป็น 3NF:

- DEPARTMENT(DeptID, DeptName)
- EMPLOYEE(EmployeeID, EmployeeName, EmpDeptID (FK), HireYear)
- COURSE(CourseID, CourseName, CourseDeptID (FK), Hours)
- TRAINING(EmployeeID (FK), CourseID (FK), Result)

SPECIAL SCHEMA 4:

EMPLOYEE (Ssn, Fname, Minit, Lname, Bdate, Address, Sex, Salary, Super_ssn, Dno)
DEPARTMENT (Dnumber, Dname, Mgr_ssn, Mgr_start_date)
DEPT_LOCATIONS (Dnumber, Dlocation)
PROJECT (Pnumber, Pname, Plocation, Dnum)
WORKS_ON (Essn, Pno, Hours)
DEPENDENT (Essn, Dependent_name, Sex, Bdate, Relationship)

- หาชื่อและนามสกุลของพนักงานที่ทำงานในโปรเจกต์ชื่อ 'ProductX'

**RA:** $\pi_{\text{Fname, Lname}}$ (($\sigma_{\text{Pname='ProductX'}}$ (PROJECT)) ⋈ WORKS_ON ⋈ EMPLOYEE)

**TRC:** {t.Fname, t.Lname | t∈EMPLOYEE ∧ ∃w, p(w∈WORKS_ON ∧ p∈PROJECT ∧ p.Pname='ProductX' ∧ p.Pnumber=w.Pno ∧ w.Essn=t.Ssn)}

**DRC:** {fn, ln | ∃ s (EMPLOYEE(s, fn, _, ln, _, _, _, _, _, _) ∧ ∃ pn (PROJECT(pn, 'ProductX', _, _) ∧ WORKS_ON(s, pn, _))))}

**SQL:** SELECT E.Fname, E.Lname FROM EMPLOYEE E, WORKS_ON W, PROJECT P WHERE E.Ssn = W.Essn AND W.Pno = P.Pnumber AND P.Pname = 'ProductX';

- หาชื่อและนามสกุลของพนักงานที่ทำงานในทุก ๆ โปรเจกต์ ที่ควบคุมโดยแผนก 5

**RA:** $\pi_{\text{Fname, Lname}}$ (EMPLOYEE ⋈ ($\pi_{\text{Essn, Pno}}$ (WORKS_ON) ÷ $\pi_{\text{Pnumber}}$ ($\sigma_{\text{Dnum=5}}$ (PROJECT))))

**TRC:** {e.Fname, e.Lname | e∈EMPLOYEE ∧ ∀p∈ PROJECT(p.Dnum = 5 → ∃ w ∈ WORKS_ON (w.Pno = p.Pnumber ∧ w.Essn = e.Ssn)) }

**DRC:** {FN, LN | ∃ Ssn, ... (EMPLOYEE(Ssn, FN, ...) ∧ ∀ Pno, ... (¬PROJECT(Pno, ..., 5) ∨ ∃ Hrs(WORKS_ON(Ssn, Pno, Hrs))))}

**SQL:** SELECT Fname, Lname FROM EMPLOYEE E WHERE NOT EXISTS ( SELECT Pnumber FROM PROJECT P WHERE Dnum = 5 AND NOT EXISTS ( SELECT 1 FROM WORKS_ON W WHERE W.Pno = P.Pnumber AND W.Essn = E.Ssn ) );

- หาชื่อและนามสกุลของพนักงานที่ไม่มีผู้อยู่ในอุปการะ

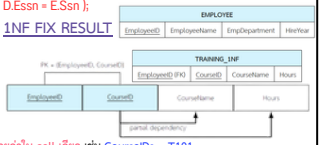
**RA:** $\pi_{\text{Fname, Lname}}$ (EMPLOYEE ⋈ ($\pi_{\text{Ssn}}$ (EMPLOYEE) − $\pi_{\text{Essn}}$ (DEPENDENT)))

**TRC:** {e.Fname, e.Lname | e∈EMPLOYEE ∧ ¬(∃d∈DEPENDENT (d.Essn = e.Ssn)) }

**DRC:** {FN, LN | ∃ Ssn, ... (EMPLOYEE(Ssn, FN, ...) ∧ ¬(∃ Dname, ... (DEPENDENT(Ssn, Dname, ...))))}

**SQL:** SELECT Fname, Lname FROM EMPLOYEE E WHERE NOT EXISTS ( SELECT 1 FROM DEPENDENT D WHERE D.Essn = E.Ssn );

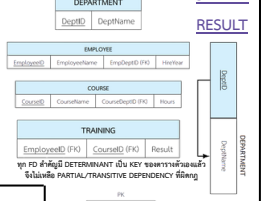
**1NF FIX RESULT**




**2NF FIX RESULT**




**3NF FIX RESULT**




## RECAP 1: Query Languages, RA, TRC, DRC

- Relational Algebra (RA): เป็นภาษา Procedural เพราะต้องบอกขั้นตอนว่าจะ Select, Project, Join อย่างไร
- SQL, TRC, DRC: เป็นภาษา Non-procedural เพราะบอกว่า "อยากได้ข้อมูลแบบไหน" มากกว่าบอกขั้นตอนทำงาน
- TRC: ใช้ตัวแปรแทน tuple ทั้งแถว เช่น t.EmpID
- DRC: ใช้ตัวแปรแทนค่าใน domain หรือแต่ละ column
- เจอคำว่า at least = ∃ หรือ EXISTS

เจอคำว่า all = ∀ หรือโครงสร้าง double NOT EXISTS

- Query แนว Division: เช่น "ลูกค้าซื้อสินค้าทุกชิ้น" หรือ "นักเรียนลงทุกวิชา" ให้คิดว่า "ไม่มี item ไหนที่ขาดไป"
- Cartesian Product: จำนวนแถวโตมาก เพราะจับคู่ทุกแถวกับทุกแถว ยิ่งต้องมีเงื่อนไข Join ที่ถูกต้องเสมอ

## RECAP 2: Transaction, ACID, Concurrency

- Predicate Pushdown: ดันเงื่อนไขกรองแถวไปทำก่อน Join เพื่อลดจำนวน tuple ที่ต้องประมวลผล
- Projection Pushdown: เลือกเฉพาะ column ที่จำเป็นให้เร็วที่สุด เพื่อลด data transfer และลด intermediate result
- ห้ามใช้ SELECT *: เพราะดึง column เกินจำเป็น ทำให้ I/O สูง และลดโอกาสที่ optimizer จะ optimize ได้ดี
- WHERE vs HAVING: WHERE กรองก่อน GROUP BY ส่วน HAVING กรองหลัง Aggregate แล้ว
- Index: ช่วยให้ SELECT เร็วขึ้น แต่ทำให้ INSERT / UPDATE / DELETE ช้าลง เพราะต้องอัพเดต index เพิ่ม
- Composite Index: ต้องคิดลำดับ column ให้ตรงกับเงื่อนไขค้นหา หลัก เช่นค้นด้วย DeptID ก่อน Salary ก็ควรมี index เริ่มจาก DeptID
- Join Order: ตารางที่ถูกกรองแล้วเหลือน้อยควรถูก Join ก่อน เพื่อลด intermediate result
- Horizontal Partitioning: แบ่งตารางตามแถว เช่น แบ่งลูกค้าตามช่วง A-Z หรือแบ่ง order ตามปี ส่วน Vertical Partitioning: แบ่งตารางตาม column และ Replication: ทำสำเนาข้อมูลหลายชุด ช่วย reliability และ availability แต่ต้องระวัง consistency

## RECAP 3: Query Optimization

- Atomicity: transaction ต้องสำเร็จทั้งก้อน หรือ rollback ทั้งก้อน ไม่มีครึ่ง ๆ กลาง ๆ
- Consistency: ก่อนและหลัง transaction ฐานข้อมูลต้องไม่ละเมิด constraint
- Isolation: transaction ที่รันพร้อมกันต้องไม่รบกวนกันจนผลลัพธ์ผิด
- Durability: commit แล้วข้อมูลต้องอยู่ แม้ระบบล่ม
- Dirty Read: อ่านค่าที่ยังไม่ commit
- Non-repeatable Read: อ่าน row เดิมสองครั้งแล้วค่าเปลี่ยน
- Phantom Read: query เงื่อนไขเดิมสองรอบ แล้วรอบหลังมีแถวใหม่โผล่
- Isolation Level ยิ่งสูง ยิ่งกันปัญหา concurrency ได้ดี แต่ performance ลดลงและ lock มากขึ้น
- Serializable: isolation สูงสุด ผลเหมือน transaction รันเรียงทีละตัว
- SAVEPOINT: ตั้งจุด rollback ย่อยใน transaction
- SELECT FOR UPDATE: lock row ที่อ่าน เพื่อเตรียมแก้ไข
- Autocommit: แต่ละ SQL statement ถูก commit อัตโนมัติ ถ้าอยาก rollback หลายคำสั่งพร้อมกันต้องเริ่ม transaction เอง

## RECAP 4: Advanced SQL, Trigger, View, Security

- Trigger: logic ที่ทำงานอัตโนมัติเมื่อเกิด event เช่น INSERT, UPDATE, DELETE
- BEFORE Trigger: เหมาะกับ validate หรือแก้ค่าก่อนบันทึก
- AFTER Trigger: เหมาะกับ audit log หรือ action หลังบันทึกสำเร็จ
- Assertion: constraint ที่ตรวจได้ข้าม table แต่ DBMS หลายตัวไม่รองรับเต็มรูปแบบ มักต้องใช้ trigger หรือ application logic แทน
- GRANT: ให้สิทธิ์, REVOKE: ถอนสิทธิ์, หลักการให้สิทธิ์: ให้เท่าที่จำเป็น ไม่ให้กว้างเกินงาน
- View: virtual table ใช้ซ่อน column สำคัญ เช่น salary หรือทำหน้าจอข้อมูลให้อ่านง่ายขึ้น
- Updatable View: ถ้าอยาก update กลับ table จริงได้ view ควรเรียบง่าย อิง table เดียว และไม่ควรมี aggregate, GROUP BY, DISTINCT หรือ join ซับซ้อน

## RECAP 5: NoSQL and MongoDB

- NoSQL: Not Only SQL เหมาะกับงาน schema ยืดหยุ่น scale ง่าย หรือข้อมูลมีหลายรูปแบบ
- MongoDB Structure: Database → Collection → Document → Field
- MongoDB เก็บข้อมูลแบบ BSON ซึ่งคล้าย JSON แต่รองรับชนิดข้อมูลเพิ่มและเหมาะกับการเก็บจริงใน database
- JSON: key และ string ต้องใช้ double quotes และห้ามมี trailing comma
- Document DB: เหมาะกับข้อมูล nested object หรือ array
- Key-value DB: เหมาะกับ cache หรือ session
- Graph Database: เหมาะกับข้อมูลที่ relationship เยอะ เช่น social network หรือ recommendation
- Mongoose: เป็น ODM ฝั่ง Node.js ใช้กำหนด schema/model และช่วยคุยกับ MongoDB แบบ object

## RECAP 6: NoSQL and MongoDB

- OLTP: เน้น transaction สั้น ๆ จำนวนมาก เช่น insert order, update stock, transfer money
- OLAP: เน้น query วิเคราะห์ข้อมูลก้อนใหญ่ เช่น report, dashboard, trend, aggregation
- Operational Database: ใช้กับงานประจำวันและข้อมูลปัจจุบัน
- Data Warehouse: เก็บข้อมูลประวัติที่จัดรูปแล้ว เหมาะกับ BI และการวิเคราะห์ระยะยาว
- Data Lake: เก็บข้อมูล raw หลายรูปแบบ ใช้แนวคิด schema-on-read คือค่อยตีความ schema ตอนนำไปใช้
- ถ้าโจทย์เป็น clean data ใช้ทำ BI → Data Warehouse
- ถ้าโจทย์บอก raw, diverse, future analytics → Data Lake
- CAP Theorem: distributed system ต้อง trade-off ระหว่าง Consistency, Availability, Partition Tolerance
- Elasticity: เพิ่มลด resource ตาม demand ได้ เหมาะกับ cloud workload ที่ traffic ขึ้นลงไม่คงที่

## RECAP 7: Normalization

- 1NF: ทุกช่องต้องเป็น atomic ห้ามมี list, repeating group หรือหลายค่าใน cell เดียว
- 2NF: ตรวจเฉพาะกรณี primary key เป็น composite key ห้าม non-prime attribute พึ่งพาแค่บางส่วนของ key
- 3NF: ห้าม non-key attribute ไปกำหนด non-key attribute ด้วยกันเอง
- Partial Dependency: attribute ธรรมดาขึ้นกับแค่บางส่วนของ composite key

Transitive Dependency: key → non-key → non-key

- เวลาทำข้อสอบ Normalization: ขีดเส้นใต้ PK ก่อน แล้วลอง FK หรือเขียนกำกับ จากนั้นค่อยลาก FD ว่าใครกำหนดใคร
- อย่าเชื่อตารางที่ดูเหมือน normalize แล้ว ต้องถามว่า attribute นี้ขึ้นกับ key ทั้งก้อนจริงไหม
- อีกคำถามสำคัญ: determinant ฝั่งซ้ายเป็น key ของตารางนั้นหรือเปล่า
- Pattern หลอกบ่อย: ตารางกลาง composite key แต่ยังเก็บชื่อพนักงาน ชื่อโปรเจกต์ ที่อยู่ลูกค้า หรือชื่อแผนกไว้ด้วย
- หลังแก้ 3NF: table หลักควรเก็บ entity, table กลางควรเก็บ relationship, และ FK ใช้ไยงกลับโดยไม่ duplicate ชื่อที่ lookup ได้

SCHEMA 2:

Student(SID, SName, Major, Year)
Course(CID, CName, Credits, Dept)
Register(SID, CID, Grade)

- Students who register at least one course

**RA:** $\pi_{\text{SID, SName}}$(Student ⋈ Register)

**TRC:** {s | s∈Student ∧ ∃r∈Register(r.SID = s.SID)}

**DRC:** {sID, sN, M, Y | Student(sID, sN, M, Y) ∧ ∃ cID, G (Register(sID, cID, G))}

**SQL:** SELECT DISTINCT s.* FROM Student s JOIN Register r ON s.SID = r.SID;

- Students who register no course

**RA:** $\pi_{\text{SID}}$(Student) − $\pi_{\text{SID}}$(Register)

**TRC:** {s | s∈Student ∧ ¬∃r∈Register(r.SID = s.SID)}

**DRC:** {sID, sN, M, Y | Student(sID, sN, M, Y) ∧ ¬∃ cID, G (Register(sID, cID, G))}

**SQL:** SELECT * FROM Student WHERE SID NOT IN (SELECT SID FROM Register);

- Students who register "Database Systems"

**RA:** $\pi_{\text{Student.*}}$ (Student ⋈ Register ⋈ $\sigma_{\text{CName='Database Systems'}}$ (Course))

**TRC:** {s | s∈Student ∧ ∃r∈Register, c∈Course (r.SID = s.SID ∧ r.CID = c.CID ∧ c.CName = 'Database Systems')}

**DRC:** {sID, sN, M, Y | Student(sID, sN, M, Y) ∧ ∃ cID, G, cR, D (Register(sID, cID, G) ∧ Course(cID, 'Database Systems', cR, D))}

**SQL:** SELECT s.* FROM Student s JOIN Register r ON s.SID = r.SID JOIN Course c ON r.CID = c.CID WHERE c.CName = 'Database Systems';

- Students who get only A grades

**RA:** $\pi_{\text{SID}}$($\sigma_{\text{Grade='A'}}$ (Register)) − $\pi_{\text{SID}}$($\sigma_{\text{Grade<>'A'}}$ (Register))

**TRC:** {s | s∈Student ∧ ∃r1∈Register (r1.SID = s.SID ∧ r1.Grade = 'A') ∧ ¬∃r2∈Register (r2.SID = s.SID ∧ r2.Grade <> 'A')}

**DRC:** {sID, sN, M, Y | Student(sID, sN, M, Y) ∧ ∃ cID1, G1 (Register(sID, cID1, 'A') ∧ ¬∃ cID2, G2 (Register(sID, cID2, G2) ∧ G2<>'A'))}

**SQL:** SELECT s.* FROM Student s WHERE SID IN (SELECT SID FROM Register WHERE Grade = 'A') AND SID NOT IN (SELECT SID FROM Register WHERE Grade <> 'A');

SCHEMA 3:

Product(PID, PName, Price)
Order(OID, OrderDate)
OrderItem(OID, PID, Quantity)
Payment(OID, PayDate, Amount)

- Products ordered today

**RA:** $\pi_{\text{Product.*}}$ (Product ⋈ OrderItem ⋈ $\sigma_{\text{OrderDate=today}}$ (Order))

**TRC:** {p | p∈Product ∧ ∃ oi∈OrderItem, o∈Order (oi.PID = p.PID ∧ oi.OID=o.OID ∧ o.OrderDate=CURRENT_DATE)}

**DRC:** {PID, PN, PR | Product(PID, PN, PR) ∧ ∃OID, Q (OrderItem(OID, PID, Q) ∧ Order(OID, CURRENT_DATE))}

**SQL:** SELECT DISTINCT p.* FROM Product p JOIN OrderItem oi ON p.PID = oi.PID JOIN Order o ON oi.OID = o.OID WHERE o.OrderDate = CURRENT_DATE;

- Customers/orders with multiple products

**RA:** $\pi_{\text{oi1.OID}}$ ($\sigma_{\text{oi1.PID <> oi2.PID}}$ ($\rho_{\text{oi1}}$ (OrderItem) ⋈$_{\text{oi1.OID=oi2.OID}}$ $\rho_{\text{oi2}}$(OrderItem)))

**TRC:** {o | o∈Order ∧ ∃ oi1, oi2∈OrderItem (oi1.OID = o.OID ∧ oi2.OID = o.OID ∧ oi1.PID <> oi2.PID) }

**DRC:** {OID,OD | Order(OID, OD)∧ ∃P1, Q1, P2, Q2 (OrderItem(OID, P1, Q1)∧OrderItem(OID, P2, Q2)∧P1<>P2)}

**SQL:** SELECT OID FROM OrderItem GROUP BY OID HAVING COUNT(PID) > 1;

- Orders where total payment < total order cost

**SQL:** SELECT t1.OID FROM ( SELECT oi.OID, SUM(oi.Quantity * p.Price) AS Cost FROM OrderItem oi JOIN Product p ON oi.PID = p.PID GROUP BY oi.OID ) t1 JOIN ( SELECT OID, SUM(Amount) AS Paid FROM Payment GROUP BY OID ) t2 ON t1.OID = t2.OID WHERE t2.Paid < t1.Cost;

- (Challenge) Courses that all students register for

**RA:** $\pi_{\text{CID, SID}}$(Register) ÷ $\pi_{\text{SID}}$(Student)

**TRC:** {c | c∈Course ∧ ∀s∈Student ( ∃r∈Register (r.SID = s.SID ∧ r.CID = c.CID))}

**DRC:** {cID, cN, cR, D | Course(cID, cN, cR, D) ∧ ∀sID, sN, M, Y (Student(sID, sN, M, Y) → ∃G(Register(sID, cID, G)))}

**SQL:** SELECT c.* FROM Course c WHERE NOT EXISTS (SELECT SID FROM Student EXCEPT SELECT SID FROM Register WHERE CID = c.CID);

- (Challenge) Products ordered in every order

**RA:** $\pi_{\text{PID, OID}}$(OrderItem) ÷ $\pi_{\text{OID}}$ (Order)

**TRC:** {p | p∈Product ∧ ∀o∈Order ( ∃ oi∈OrderItem (oi.PID = p.PID ∧ oi.OID = o.OID))}

**DRC:** {PID, PN, PR | Product(PID, PN, PR) ∧ ∀OID, OD (Order(OID, OD) → ∃Q(OrderItem(OID, PID, Q)))}

**SQL:** SELECT p.* FROM Product p WHERE NOT EXISTS (SELECT OID FROM Order EXCEPT SELECT OID FROM OrderItem WHERE PID = p.PID);

SCHEMA 1:

Employee(EID, EName, Dept, Salary, ManagerID)

- Employees whose manager works in the same department

**RA:** $\pi_{e}$-(Employee e ⋈ $_{e.ManagerID=m.EID \wedge e.Dept=m.Dept}$ Employee m)

**TRC:** {e | e∈Employee ∧ ∃m∈Employee (e.ManagerID = m.EID ∧ e.Dept = m.Dept)}

**DRC:** {eID, eN, d, s, mID | Employee(eID, eN, d, s, mID) ∧ ∃ meN, ms, mmID(Employee(mID, meN, d, ms, mmID))}

**SQL:** SELECT e.* FROM Employee e JOIN Employee m ON e.ManagerID = m.EID WHERE e.Dept = m.Dept;

- Highest-paid employee(s)

**RA:** Employee − $\pi_{e1.*}$ (Employee e1 ⋈ $_{e1.Salary < e2.Salary}$ Employee e2)

**TRC:** {e | e∈Employee ∧ ∀x∈Employee (e.Salary ≥ x.Salary)}

**DRC:** {eID, eN, d, s, mID | Employee(eID, eN, d, s, mID) ∧ ∀xID, xN, xd, xs, xmID(Employee(xID, xN, xd, xs, xmID)→s≥xs)}

**SQL:** SELECT * FROM Employee WHERE Salary >= ALL (SELECT Salary FROM Employee);

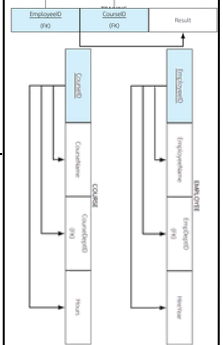
- (Challenge) Managers who manage all employees in their department

**RA:** $\pi_{\text{ManagerID, Dept}}$ (Employee) ÷ $\pi_{\text{EID, Dept}}$ (Employee)

**TRC:** {m | m∈Employee ∧ ∀e∈Employee (e.Dept = m.Dept → e.ManagerID = m.EID) }

**DRC:** {mID, mN, md, ms, mmID | Employee(mID, mN, md, ms, mmID) ∧ ∀eID, eN, s, m2 (Employee(eID, eN, md, s, m2) → m2 = mID)}

**SQL:** SELECT * FROM Employee m WHERE NOT EXISTS (SELECT EID FROM Employee e WHERE e.Dept = m.Dept EXCEPT SELECT EID FROM Employee WHERE ManagerID = m.EID);




**Recap Normalization**

- 1NF (Atomic): ค่าในแต่ละช่องต้องแบ่งย่อยไม่ได้อีก (ห้ามมีหลายค่าในช่องเดียว / ห้ามเป็น Array) + ต้องมี Primary Key (PK)
- 2NF (No Partial Dependency): ต้องเป็น 1NF + ห้ามขึ้นกับส่วนใดส่วนหนึ่ง ของ PK (เกิดเฉพาะตอนมี Composite PK 2 ตัวขึ้นไป เช่น ถ้าคอลัมน์ A,B เป็น PK คอลัมน์อื่นต้องขึ้นกับทั้ง A และ B ห้ามขึ้นกับแค่ A ตัวเดียว) **วิธีแก้:** แตกตารางใหม่ให้คอลัมน์นั้นไปอยู่กับ PK ของมัน
- 3NF (No Transitive Dependency): ต้องเป็น 2NF + ห้ามมี Non-key ชี้ไปหา Non-key ด้วยกันเอง (เช่น รหัสสต(PK) -> รหัสคณะ -> ชื่อคณะ แบบนี้ ชื่อคณะ ขึ้นกับรหัสคณะ ซึ่งไม่ใช่ PK) **วิธีแก้:** แยกตารางคณะออกมาต่างหาก แล้วเอา รหัสคณะ เป็น FK
- BCNF (Boyce-Codd Normal Form): ต้องเป็น 3NF + ตัวที่ไปกำหนดค่าคนอื่น (Determinant) ต้องเป็น Candidate Key เสมอ (เข้มงวดกว่า 3NF: ห้าม Non-key ย้อนกลับไปกำหนดส่วนใดส่วนหนึ่งของ PK)